

UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ciencias

Escuela Profesional de Ciencia de la Computación

- **Curso: Fundamentos de Programación**
- **Docente:** Américo Chulluncuy Reynoso

2026-1

Sesión 11:

Gestión dinámica de memoria y estructuras

Contenido

1. El problema: ¿por qué no alcanza con arreglos?
2. Esctruturas: repaso
3. Estructuras autoreferenciadas
4. Visualización de memoria: nodo enlazado vs arreglo
5. Lista Enlazada Simple:
6. Árbol Binario

1. ¿Cómo manejamos datos que cambian constantemente?

🎵 **Spotify** — Una playlist puede tener 0 canciones o 10,000. El usuario agrega y elimina en tiempo real.

🌐 **Tu navegador** — El historial de "atrás/adelante" crece y decrece con cada clic. Nadie sabe cuántas páginas visitarás.

🖨️ **Cola de impresión** — Los documentos llegan en cualquier orden y cantidad. El sistema los encola y desencola dinámicamente.

int playlist[???];
¿cuánto reservo?
Si reservo poco → se rompe
Si reservo mucho → desperdicio
vs
[nodo]→[nodo]→[nodo]→...
crece cuando necesita,
muere (libera) cuando sobra

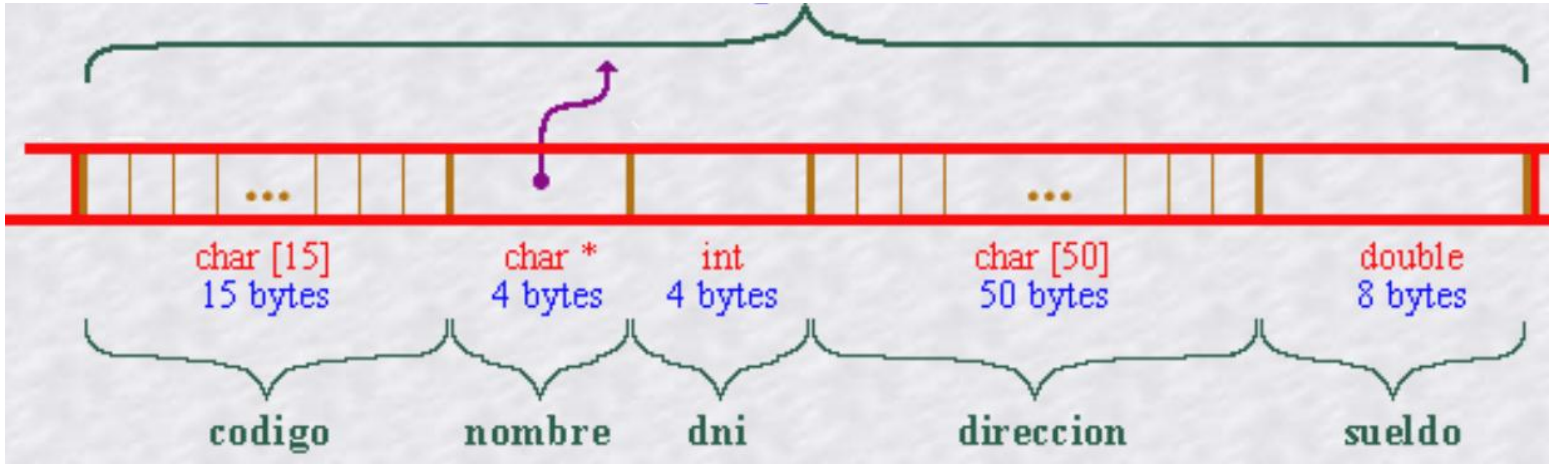
¿Qué estructura permite insertar o eliminar en cualquier posición sin mover ningún otro dato?

2. Estructuras: repaso

- Una variable de tipo struct se almacena en la pila, como se muestra a continuación:

```
struct TPersona {
        char codigo[15];
        char *nombre;
        int dni;
        char direccion[50];
        double sueldo;
};
```

TPersona empleado;

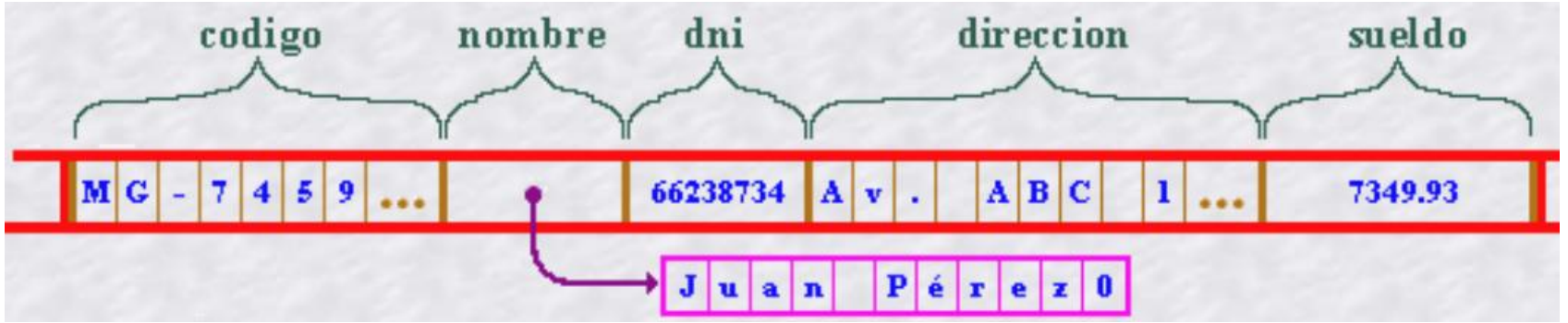


- Aunque la operación de asignación se puede dar entre variables de tipo struct, hay que tener cuidado cuando asignamos datos a sus miembros.

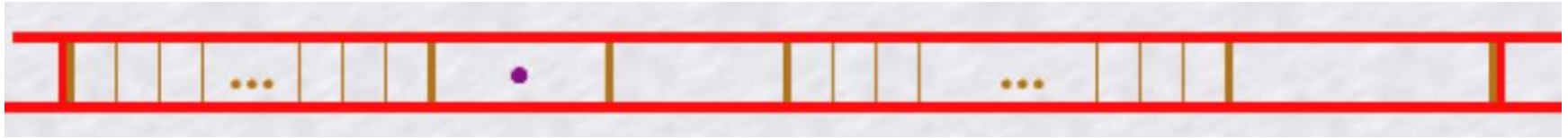
Estructura: Asignación de datos

Antes de la asignación:

`empleado`

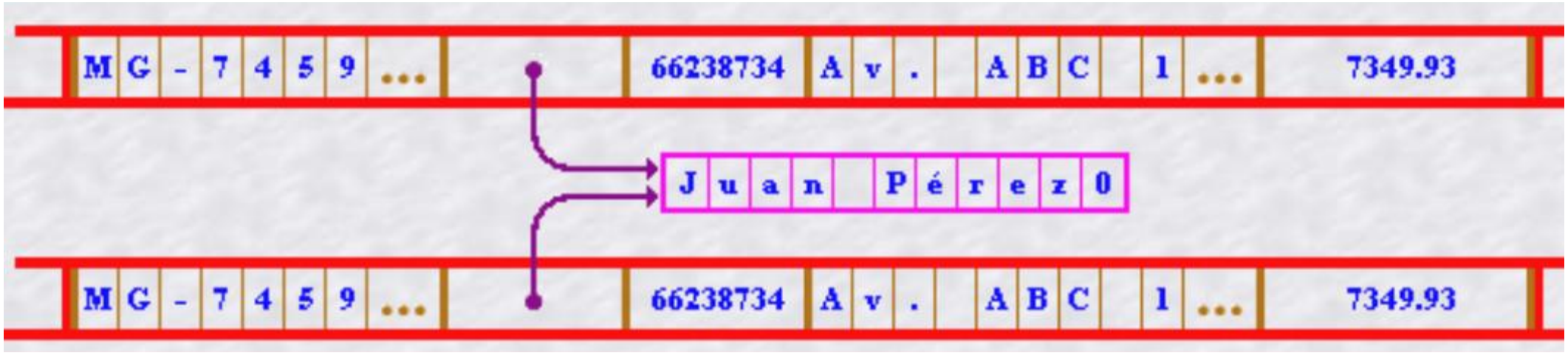


`trabajador`



Estructura: Asignación de datos

Luego de la asignación: `trabajador = empleado;`

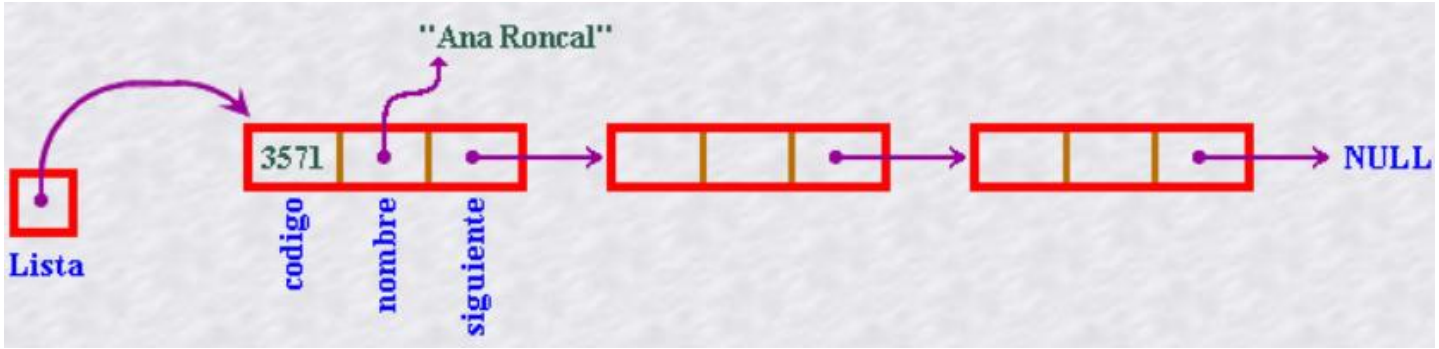


3. Estructuras autoreferenciadas

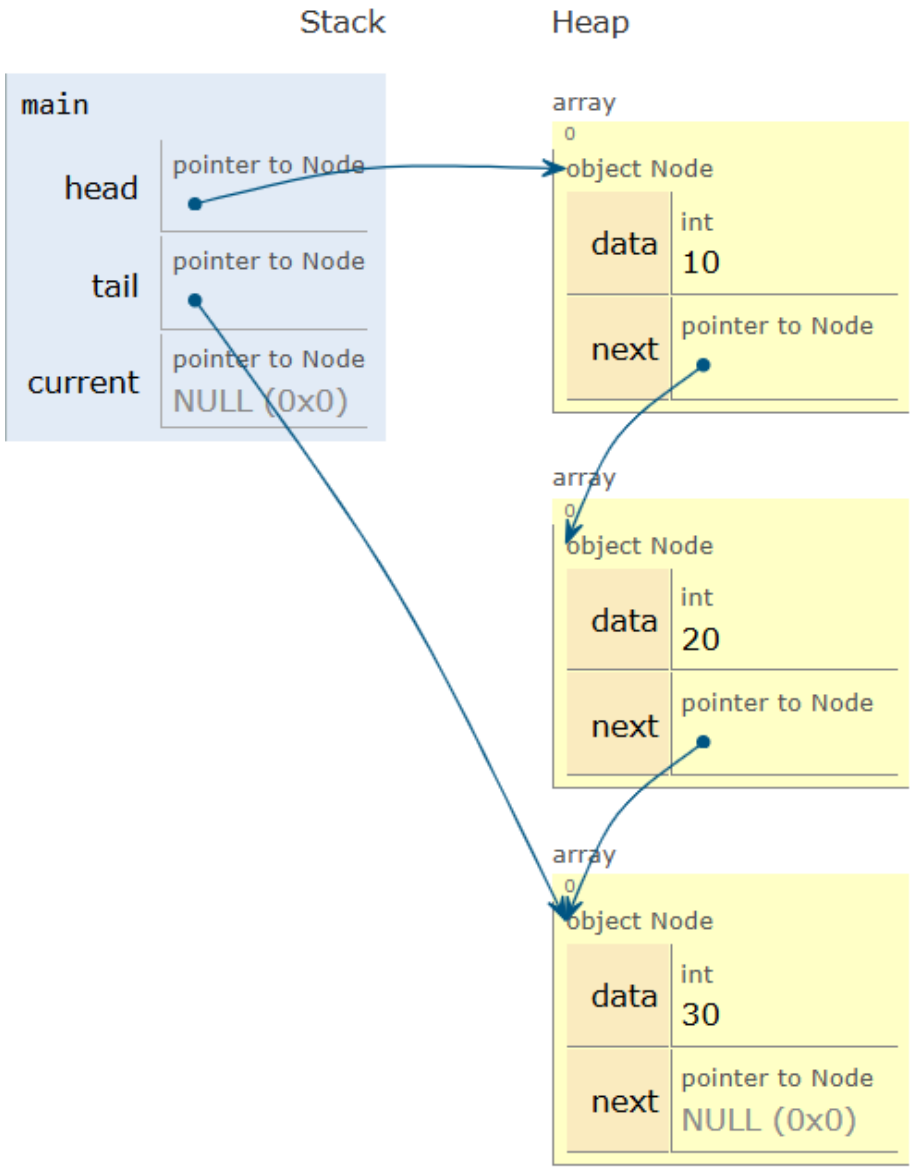
- Una estructura autoreferenciada es una variable de tipo struct en la que uno o más de sus miembros (campos) son punteros del mismo tipo que la estructura que lo define.

- Bajo este concepto, una variable definida así puede enlazarse (apuntar), a través de estos miembros, a otra u otras variables del mismo tipo, generando listas enlazadas (simples, doblemente enlazada, circulares), pilas, colas, árboles, etc.

```
struct NodoLSL {
    int codigo;
    char *nombre;
    NodoLSL *siguiente;
};
```



4. Visualización en memoria: nodo enlazado vs arreglo



5. Listas enlazadas

```
struct Nodo {
    int data;
    Nodo* next;
};
```

6. Arbol binario

```
struct Nodo {
    int data;
    Nodo* left;
    Nodo* right;
};
```